The approximation failure mode: convolutional networks

AI Methods for Science – Part I – Week 3, Monday

Yohai Bar-Sinai

October 25, 2026

Where we are

1. A flexible function f_θ
today: the right functions

2. A cost function $L(\theta)$
unchanged

3. An optimizer
last week's
recipe, unchanged

Last week: the training loss goes to 0 when we let it, the priors close a few points, and the MLP stops near 60% on CIFAR-10. A human: 94%.
Something about images is not in the ansatz.

Failure	Symptom
Optimization	training loss stays high
Approximation	the family cannot represent the answer, or only with far more parameters than the data can fix
Generalization	training loss low, held-out loss high

Where we are

1. A flexible function f_θ
today: the right functions

2. A cost function $L(\theta)$
unchanged

3. An optimizer
last week's recipe, unchanged

Last week: the training loss goes to 0 when we let it, the priors close a few points, and the MLP stops near 60% on CIFAR-10. A human: 94%. Something about images is not in the ansatz.

1. what an MLP does not know about an image
2. convolution in 1-D: stencils, stride, pooling, receptive field; images, weight sharing; what the filters learn
3. Thursday: borrowing a trained network; skip connections
4. the lab: E16–E19

Failure	Symptom
Optimization	training loss stays high
Approximation	the family cannot represent the answer, or only with far more parameters than the data can fix
Generalization	training loss low, held-out loss high

Convolution

What an image is to the MLP

$\mathbf{x} \in \mathbb{R}^{3072}$: a list of numbers. The first layer is $\mathbf{W}^{(1)}\mathbf{x}$, one weight per (pixel, unit).

What an image is to the MLP

$\mathbf{x} \in \mathbb{R}^{3072}$: a list of numbers. The first layer is $\mathbf{W}^{(1)}\mathbf{x}$, one weight per (pixel, unit).

- Shift the cat by one pixel: a different vector; the weights that learned the cat at $(10, 12)$ know nothing about $(11, 12)$. 1.6M first-layer weights, each used by one pixel.

What an image is to the MLP

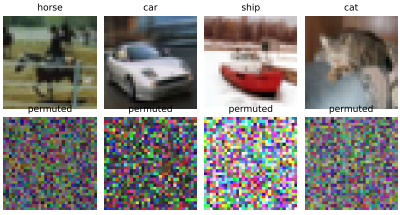
$\mathbf{x} \in \mathbb{R}^{3072}$: a list of numbers. The first layer is $\mathbf{W}^{(1)}\mathbf{x}$, one weight per (pixel, unit).

- Shift the cat by one pixel: a different vector; the weights that learned the cat at $(10, 12)$ know nothing about $(11, 12)$. 1.6M first-layer weights, each used by one pixel.
- Every "edge here" detector is relearned at every position, from the data. $45{,}000$ images are not enough.

What an image is to the MLP

$\mathbf{x} \in \mathbb{R}^{3072}$: a list of numbers. The first layer is $\mathbf{W}^{(1)}\mathbf{x}$, one weight per (pixel, unit).

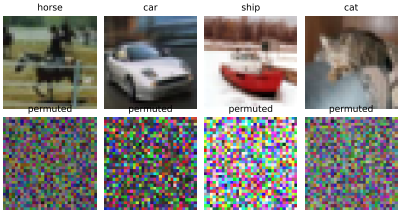
- Shift the cat by one pixel: a different vector; the weights that learned the cat at $(10, 12)$ know nothing about $(11, 12)$. 1.6M first-layer weights, each used by one pixel.
- Every "edge here" detector is relearned at every position, from the data. $45{,}000$ images are not enough.
- Permute the pixels, the same permutation for every image (right): $\mathbf{W}^{(1)}$ permutes its columns and nothing else changes. **To the MLP the two rows are the same problem.**



What an image is to the MLP

$\mathbf{x} \in \mathbb{R}^{3072}$: a list of numbers. The first layer is $\mathbf{W}^{(1)}\mathbf{x}$, one weight per (pixel, unit).

- Shift the cat by one pixel: a different vector; the weights that learned the cat at $(10, 12)$ know nothing about $(11, 12)$. 1.6M first-layer weights, each used by one pixel.
- Every "edge here" detector is relearned at every position, from the data. $45{,}000$ images are not enough.
- Permute the pixels, the same permutation for every image (right): $\mathbf{W}^{(1)}$ permutes its columns and nothing else changes. **To the MLP the two rows are the same problem.**



What we know about images, and the MLP does not:

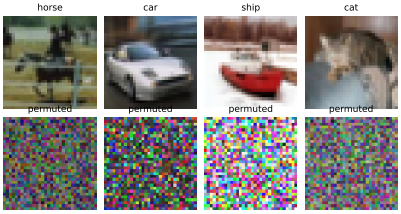
- nearby pixels matter together (*locality*)
- a feature is the same feature wherever it appears (*translation equivariance*)
- structure at every scale: edges → textures → parts → objects (*hierarchy*)

What an image is to the MLP

$\mathbf{x} \in \mathbb{R}^{3072}$: a list of numbers. The first layer is $\mathbf{W}^{(1)}\mathbf{x}$, one weight per (pixel, unit).

- Shift the cat by one pixel: a different vector; the weights that learned the cat at $(10, 12)$ know nothing about $(11, 12)$. 1.6M first-layer weights, each used by one pixel.
- Every "edge here" detector is relearned at every position, from the data. $45{,}000$ images are not enough.
- Permute the pixels, the same permutation for every image (right): $\mathbf{W}^{(1)}$ permutes its columns and nothing else changes. **To the MLP the two rows are the same problem.**

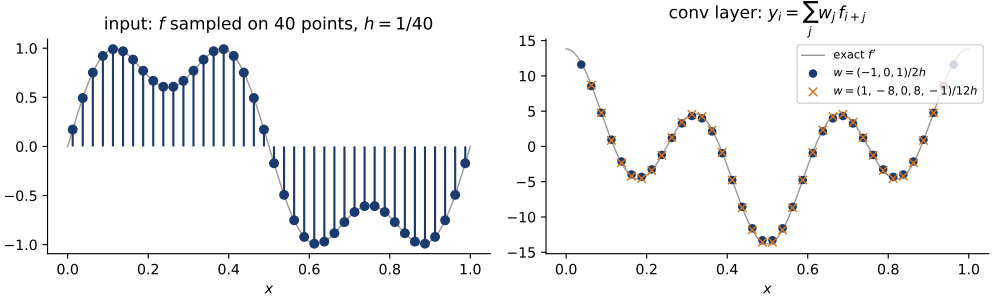
Build the prior into the ansatz (*inductive bias*); the data only supply what is left. Last week a symmetry was stated as data (augmentation); today as architecture.



What we know about images, and the MLP does not:

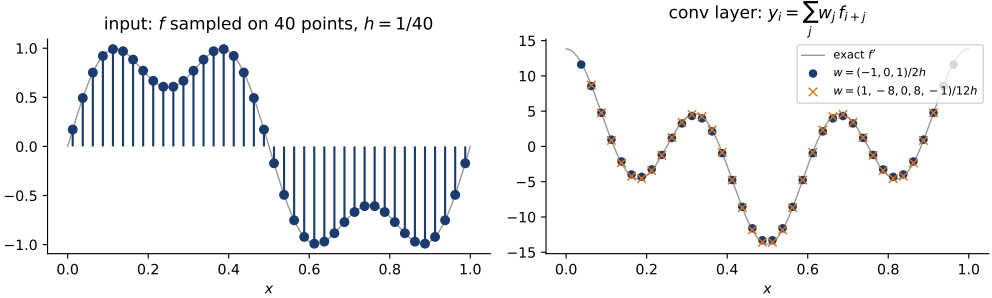
- nearby pixels matter together (*locality*)
- a feature is the same feature wherever it appears (*translation equivariance*)
- structure at every scale: edges → textures → parts → objects (*hierarchy*)

A convolutional layer, in 1-D



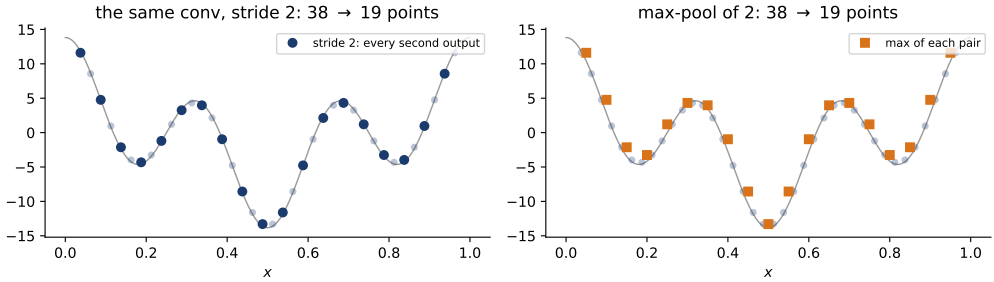
- A function on a grid, $f_i = f(x_i)$. A **conv layer** slides one small set of weights over it, $y_i = \sum_j w_j\, f_{i+j}$, *the same* w *at every position*: a **stencil**.

A convolutional layer, in 1-D



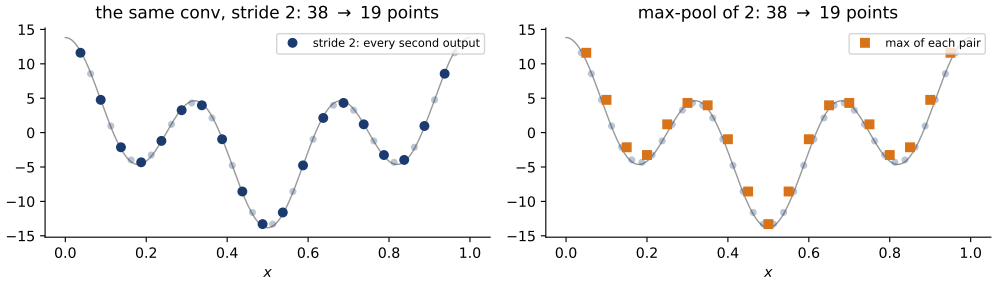
- A function on a grid, $f_i = f(x_i)$. A **conv layer** slides one small set of weights over it, $y_i = \sum_j w_j\, f_{i+j}$, *the same* w *at every position*: a **stencil**.
- $w = (-1, 0, 1)/2h$ is f'; $(1, -8, 0, 8, -1)/12h$ the fourth-order version. Kernel size $k = 3$ or 5; without padding the output is shorter by $k - 1$.

Stride and pooling: coarser grids



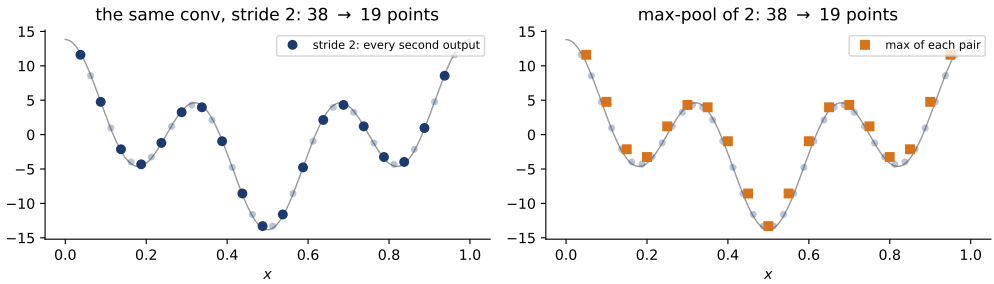
- **Stride** s: compute the conv at every s-th position only. Same weights, $s\times$ fewer outputs and $s\times$ less work.

Stride and pooling: coarser grids



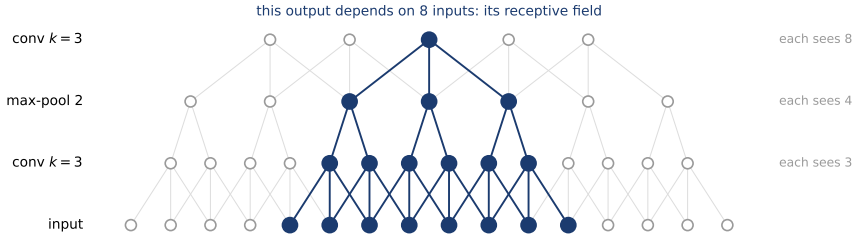
- **Stride** s: compute the conv at every s-th position only. Same weights, $s\times$ fewer outputs and $s\times$ less work.
- **Pooling**: no weights. The max (or mean) of each block of 2, then one output per block. The max keeps the strongest response wherever it was in the block.

Stride and pooling: coarser grids

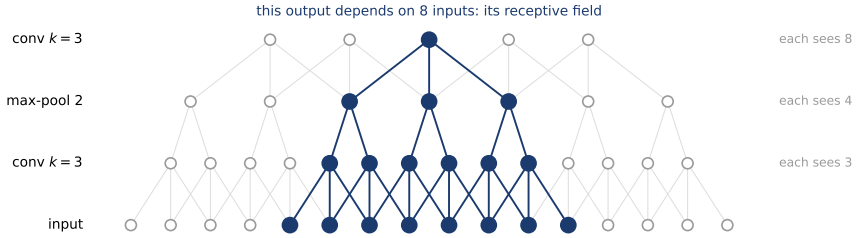


- **Stride** s: compute the conv at every s-th position only. Same weights, $s\times$ fewer outputs and $s\times$ less work.
- **Pooling**: no weights. The max (or mean) of each block of 2, then one output per block. The max keeps the strongest response wherever it was in the block.
- Either way the grid gets coarser: the next layer's stencil covers a larger piece of the signal.

The receptive field

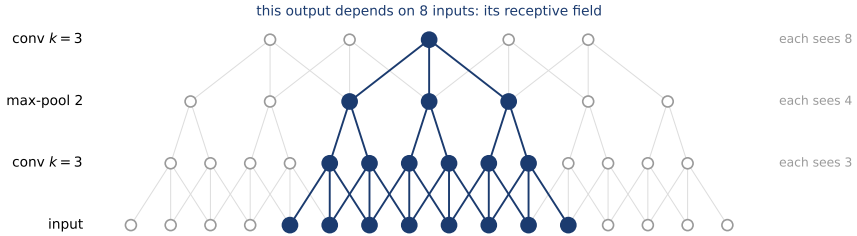


The receptive field



- Stack layers and each output depends on a growing window of the input: 3, then 4, then 8 points. Pooling doubles the step of everything after it.

The receptive field



- Stack layers and each output depends on a growing window of the input: 3, then 4, then 8 points. Pooling doubles the step of everything after it.
- Deep in the network one number summarizes a large patch, built from small stencils. This is where the hierarchy, edges $\rightarrow$ parts $\rightarrow$ objects, comes from.

Narrower and deeper

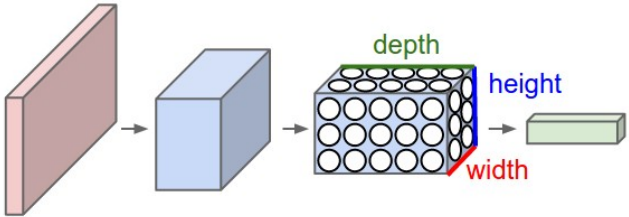
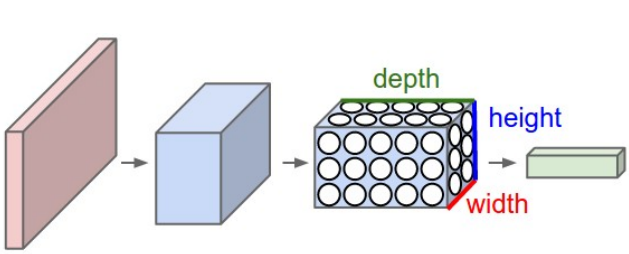


Figure: CS231n course notes, Stanford (A. Karpathy et al.).

- Every layer maps a volume to a volume: width $\times$ height $\times$ channels. Through the network the volumes get **narrower** (stride, pooling) and **deeper** (more stencils per layer).

Narrower and deeper



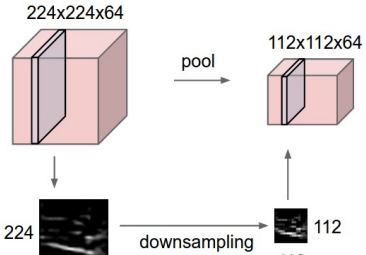
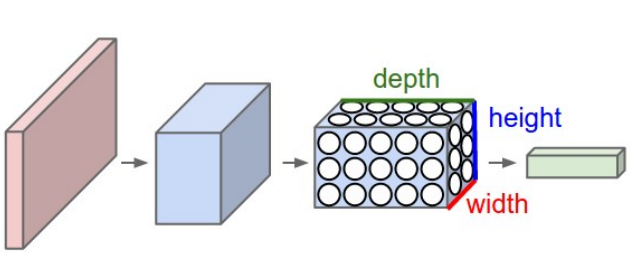


Figure: CS231n course notes, Stanford (A. Karpathy et al.).

- Every layer maps a volume to a volume: width $\times$ height $\times$ channels. Through the network the volumes get **narrower** (stride, pooling) and **deeper** (more stencils per layer).
- Pooling acts on each channel separately: $224 \times 224 \times 64 \to 112 \times 112 \times 64$. The channels are where the information goes as the grid coarsens.

Narrower and deeper



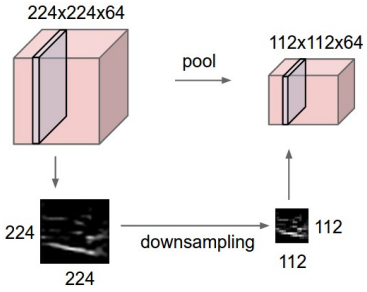
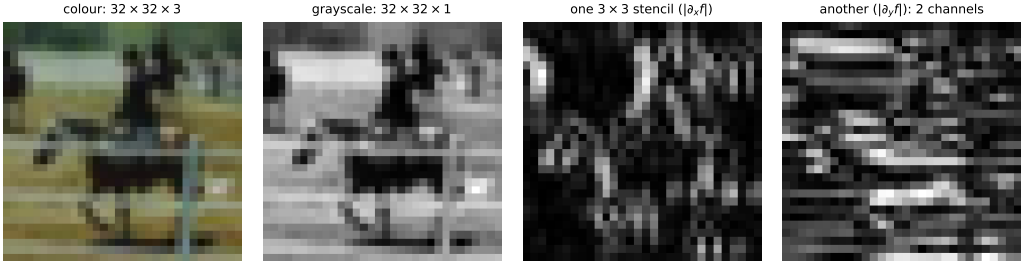


Figure: CS231n course notes, Stanford (A. Karpathy et al.).

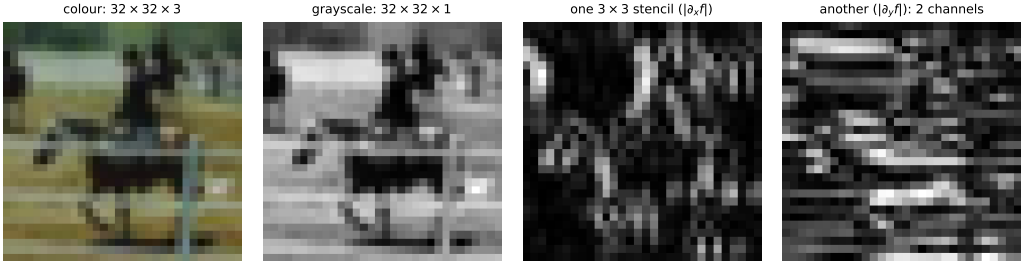
- Every layer maps a volume to a volume: width $\times$ height $\times$ channels. Through the network the volumes get **narrower** (stride, pooling) and **deeper** (more stencils per layer).
- Pooling acts on each channel separately: $224 \times 224 \times 64 \to 112 \times 112 \times 64$. The channels are where the information goes as the grid coarsens.
- At the end, a $1 \times 1 \times C$ column: one number per feature for the whole image. Then a linear layer to the classes.

From signals to images



- A grayscale image is f on a 2-D grid; the stencil is $k \times k$. C_{out} stencils give C_{out} output **channels** (feature maps).

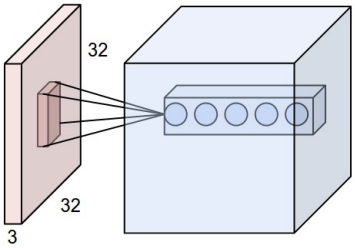
From signals to images



- A grayscale image is f on a 2-D grid; the stencil is $k \times k$. C_{out} stencils give C_{out} output **channels** (feature maps).
- Colour adds a third axis: the input is a $32 \times 32 \times 3$ **volume**; each stencil is $k \times k \times C_{\text{in}}$, summed over the input channels, one map out. The output is a volume too, $32 \times 32 \times C_{\text{out}}$.

Weight sharing

first layer	MLP	conv, $k = 3$
weights	$3072 \cdot 512 = 1.6\text{M}$	$27 \cdot 32 = 864$
uses per weight	1	1024
multiply-adds	1.6M	0.9M
whole net	1.7M	95k



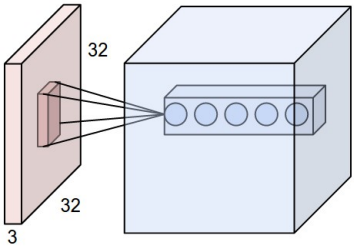
One $k \times k \times 3$ stencil, one position, one number; C_{out} of them, a column.

Figure: CS231n course notes, Stanford (A. Karpathy et al.).

Weight sharing

first layer	MLP	conv, $k = 3$
weights	$3072 \cdot 512 = 1.6\text{M}$	$27 \cdot 32 = 864$
uses per weight	1	1024
multiply-adds	1.6M	0.9M
whole net	1.7M	95k

- **Weight sharing**: one stencil weight is used at all 32×32 positions. The MLP's first-layer weight belongs to one pixel and is used once.



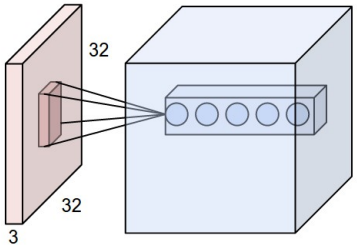
One $k \times k \times 3$ stencil, one position, one number; C_{out} of them, a column.

Figure: CS231n course notes, Stanford (A. Karpathy et al.).

Weight sharing

first layer	MLP	conv, $k = 3$
weights	$3072 \cdot 512 = 1.6\text{M}$	$27 \cdot 32 = 864$
uses per weight	1	1024
multiply-adds	1.6M	0.9M
whole net	1.7M	95k

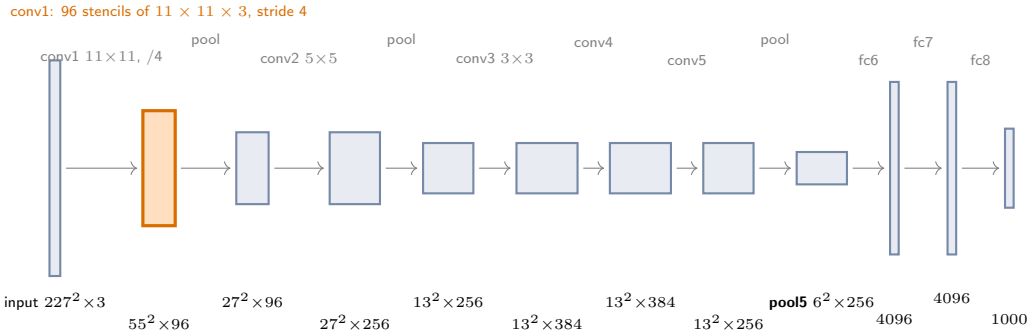
- **Weight sharing**: one stencil weight is used at all 32×32 positions. The MLP's first-layer weight belongs to one pixel and is used once.
- Same compute, $2000\times$ fewer parameters. The 1.6M numbers the MLP had to learn from $45{,}000$ images, the conv net gets from the prior: local, the same everywhere.



One $k \times k \times 3$ stencil, one position, one number; C_{out} of them, a column.

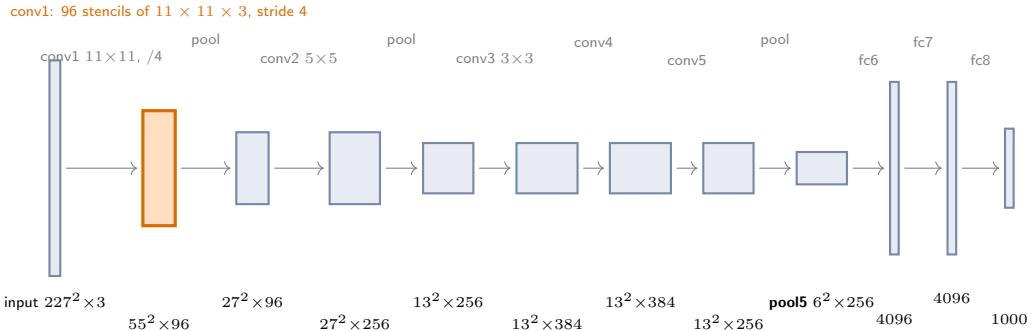
Figure: CS231n course notes, Stanford (A. Karpathy et al.).

AlexNet, and its first layer



Krizhevsky, Sutskever, Hinton 2012. 60M parameters, 58M of them in fc6–fc8.

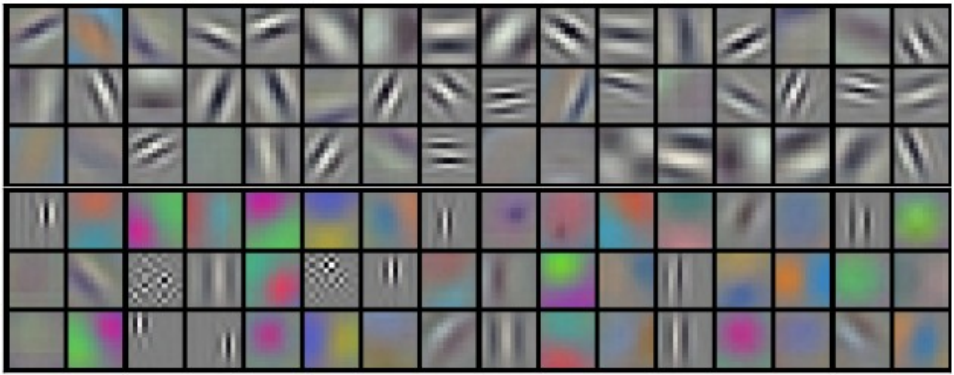
AlexNet, and its first layer



Krizhevsky, Sutskever, Hinton 2012. 60M parameters, 58M of them in fc6–fc8.

- Five conv blocks make the volume narrower and deeper, $227^2 \times 3 \to 6^2 \times 256$; three fully connected layers classify. The first layer: 96 stencils of $11 \times 11 \times 3$, stride 4. Next: what they look like after training on 1.3M images.

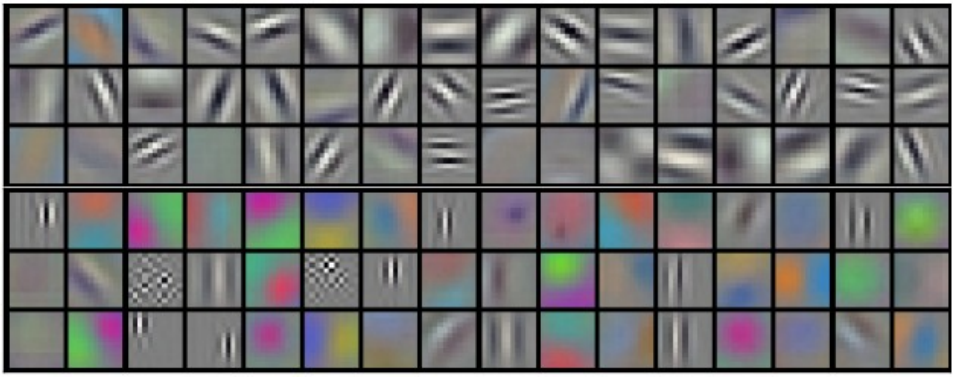
What the filters learn



The 96 first-layer stencils ($11 \times 11 \times 3$) of AlexNet, trained on ImageNet.

Krizhevsky, Sutskever, Hinton 2012, Fig. 3; via CS231n.

What the filters learn

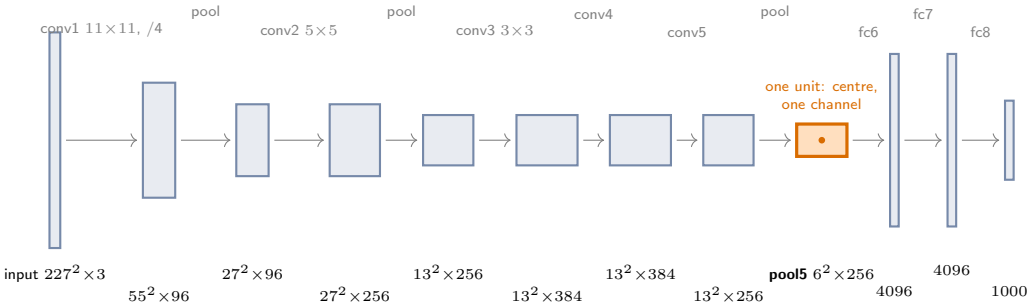


The 96 first-layer stencils ($11 \times 11 \times 3$) of AlexNet, trained on ImageNet.

Krizhevsky, Sutskever, Hinton 2012, Fig. 3; via CS231n.

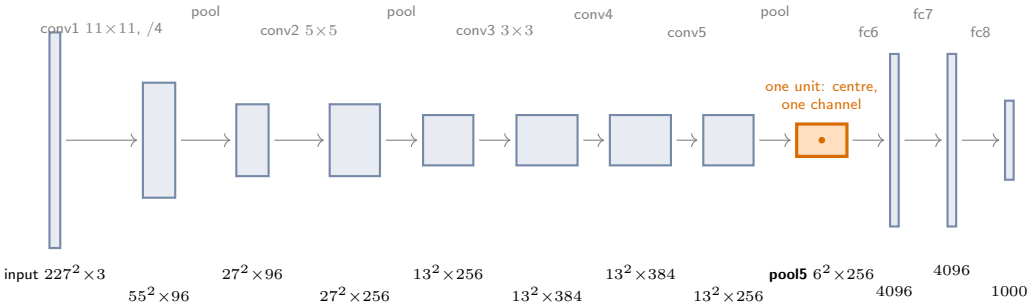
- Nobody told the network about edges: oriented edges at several scales, and colour blobs, are what the data and the prior together produce. The same stencils appear whatever the data set.

AlexNet, and where pool5 is



Krizhevsky, Sutskever, Hinton 2012.

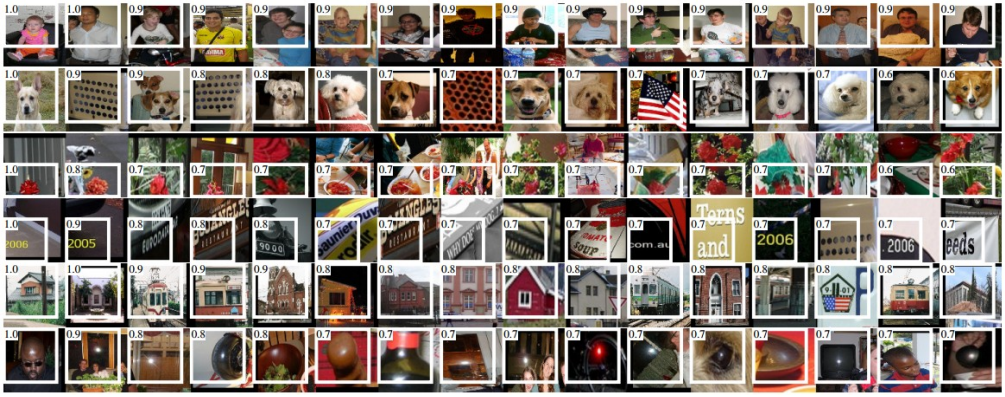
AlexNet, and where pool5 is



Krizhevsky, Sutskever, Hinton 2012.

- pool5 is the last volume, $6 \times 6 \times 256$: 256 channels, each a 6×6 map whose every entry sees 195 of the 227 input pixels. Next: what one such channel responds to.

What the deep units respond to



Six units of AlexNet's last pooling layer (pool5), one row each: of 10^7 image regions, the 16 that excite the unit most. White box: the unit's receptive field, 195×195 of the 227×227 input.

Girshick, Donahue, Darrell, Malik 2014, Fig. 4; via CS231n.

The building blocks, and the lab's network

```
nn.Sequential(
  nn.Conv2d(3, 32, 3, padding=1), nn.
       BatchNorm2d(32), nn.ReLU(),
  nn.MaxPool2d(2), # 32x32 -> 16x16
  nn.Conv2d(32, 64, 3, padding=1), nn.
       BatchNorm2d(64), nn.ReLU(),
  nn.MaxPool2d(2), # -> 8x8
  nn.Conv2d(64, 128, 3, padding=1), nn.
       BatchNorm2d(128), nn.ReLU(),
  nn.MaxPool2d(2), # -> 4x4
)
x = features(x).mean(dim=(2, 3)) # global
     average pool -> (B, 128)
logits = nn.Linear(128, 10)(x)
```

`cnn_lab.py`, `channels=[32,64,128]`: $95{,}000$ parameters. The training loop is the same five lines.

The building blocks, and the lab's network

```
nn.Sequential(
  nn.Conv2d(3, 32, 3, padding=1), nn.
       BatchNorm2d(32), nn.ReLU(),
  nn.MaxPool2d(2), # 32x32 -> 16x16
  nn.Conv2d(32, 64, 3, padding=1), nn.
       BatchNorm2d(64), nn.ReLU(),
  nn.MaxPool2d(2), # -> 8x8
  nn.Conv2d(64, 128, 3, padding=1), nn.
       BatchNorm2d(128), nn.ReLU(),
  nn.MaxPool2d(2), # -> 4x4
)
x = features(x).mean(dim=(2, 3)) # global
     average pool -> (B, 128)
logits = nn.Linear(128, 10)(x)
```

`cnn_lab.py`, `channels=[32,64,128]`: $95{,}000$ parameters. The training loop is the same five lines.

- **Block**: conv $\rightarrow$ BatchNorm $\rightarrow$ ReLU. Last week's fixes, built in. `BatchNorm2d` normalizes each channel over (batch, height, width).

The building blocks, and the lab's network

```
nn.Sequential(
  nn.Conv2d(3, 32, 3, padding=1), nn.
      BatchNorm2d(32), nn.ReLU(),
  nn.MaxPool2d(2), # 32x32 -> 16x16
  nn.Conv2d(32, 64, 3, padding=1), nn.
      BatchNorm2d(64), nn.ReLU(),
  nn.MaxPool2d(2), # -> 8x8
  nn.Conv2d(64, 128, 3, padding=1), nn.
      BatchNorm2d(128), nn.ReLU(),
  nn.MaxPool2d(2), # -> 4x4
)
x = features(x).mean(dim=(2, 3)) # global
    average pool -> (B, 128)
logits = nn.Linear(128, 10)(x)
```

`cnn_lab.py`, `channels=[32,64,128]`: $95{,}000$ parameters. The training loop is the same five lines.

- **Block**: conv $\rightarrow$ BatchNorm $\rightarrow$ ReLU. Last week's fixes, built in. `BatchNorm2d` normalizes each channel over (batch, height, width).
- **Pooling**: max over 2×2 halves the resolution, and we double the channels. Each output pixel then sees a larger patch of the input: the *receptive field* grows with depth ($3 \rightarrow 7 \rightarrow 15$ pixels here). Hierarchy.

The building blocks, and the lab's network

```
nn.Sequential(
  nn.Conv2d(3, 32, 3, padding=1), nn.
       BatchNorm2d(32), nn.ReLU(),
  nn.MaxPool2d(2), # 32x32 -> 16x16
  nn.Conv2d(32, 64, 3, padding=1), nn.
       BatchNorm2d(64), nn.ReLU(),
  nn.MaxPool2d(2), # -> 8x8
  nn.Conv2d(64, 128, 3, padding=1), nn.
       BatchNorm2d(128), nn.ReLU(),
  nn.MaxPool2d(2), # -> 4x4
)
x = features(x).mean(dim=(2, 3)) # global
     average pool -> (B, 128)
logits = nn.Linear(128, 10)(x)
```

`cnn_lab.py`, `channels=[32,64,128]`: $95{,}000$ parameters. The training loop is the same five lines.

- **Block**: conv $\rightarrow$ BatchNorm $\rightarrow$ ReLU. Last week's fixes, built in. `BatchNorm2d` normalizes each channel over (batch, height, width).
- **Pooling**: max over 2×2 halves the resolution, and we double the channels. Each output pixel then sees a larger patch of the input: the *receptive field* grows with depth ($3 \rightarrow 7 \rightarrow 15$ pixels here). Hierarchy.
- **Global average pooling**: one number per channel, wherever the feature was. Translation *invariance* at the end; no giant flatten-to-linear layer.

The building blocks, and the lab's network

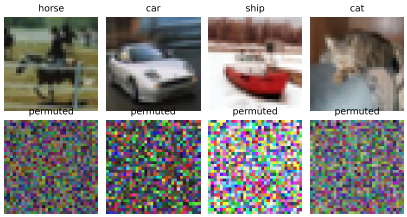
```
nn.Sequential(
  nn.Conv2d(3, 32, 3, padding=1), nn.
       BatchNorm2d(32), nn.ReLU(),
  nn.MaxPool2d(2), # 32x32 -> 16x16
  nn.Conv2d(32, 64, 3, padding=1), nn.
       BatchNorm2d(64), nn.ReLU(),
  nn.MaxPool2d(2), # -> 8x8
  nn.Conv2d(64, 128, 3, padding=1), nn.
       BatchNorm2d(128), nn.ReLU(),
  nn.MaxPool2d(2), # -> 4x4
)
x = features(x).mean(dim=(2, 3)) # global
     average pool -> (B, 128)
logits = nn.Linear(128, 10)(x)
```

`cnn_lab.py`, `channels=[32,64,128]`: $95{,}000$ parameters. The training loop is the same five lines.

- **Block**: conv $\rightarrow$ BatchNorm $\rightarrow$ ReLU. Last week's fixes, built in. `BatchNorm2d` normalizes each channel over (batch, height, width).
- **Pooling**: max over 2×2 halves the resolution, and we double the channels. Each output pixel then sees a larger patch of the input: the *receptive field* grows with depth ($3 \rightarrow 7 \rightarrow 15$ pixels here). Hierarchy.
- **Global average pooling**: one number per channel, wherever the feature was. Translation *invariance* at the end; no giant flatten-to-linear layer.
- Padding keeps the size; stride is pooling inside the conv; 1×1 convs mix channels. Arithmetic in the backup.

The lab

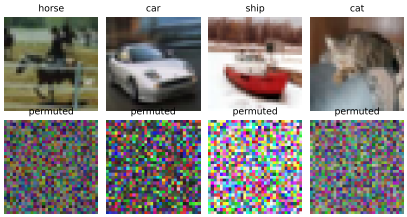
E16: what is it that the CNN knows? The permutation test



`shuffle_pixels=True`: one fixed random permutation of the 3072 pixel values, applied to every image, training and validation.

Four runs, the same recipe: the CNN and the MLP (`channels=[]`), each with and without the permutation.

E16: what is it that the CNN knows? The permutation test

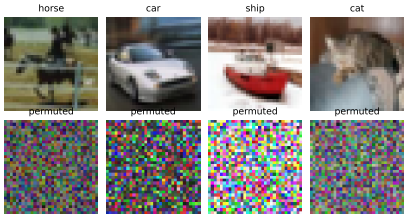


`shuffle_pixels=True`: one fixed random permutation of the 3072 pixel values, applied to every image, training and validation.

Four runs, the same recipe: the CNN and the MLP (`channels=[]`), each with and without the permutation.

- **The MLP**: the permutation relabels the input coordinates; $\mathbf{W}^{(1)}$ absorbs it. Same training problem. If it is hurt, there is a bug.

E16: what is it that the CNN knows? The permutation test

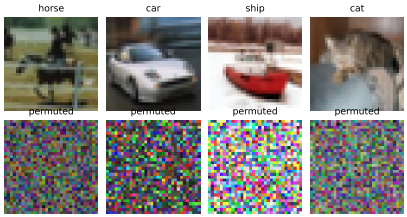


`shuffle_pixels=True`: one fixed random permutation of the 3072 pixel values, applied to every image, training and validation.

Four runs, the same recipe: the CNN and the MLP (`channels=[]`), each with and without the permutation.

- **The MLP**: the permutation relabels the input coordinates; $\mathbf{W}^{(1)}$ absorbs it. Same training problem. If it is hurt, there is a bug.
- **The CNN**: its prior (locality, equivariance) is now *false* about the data; the architecture is unchanged.

E16: what is it that the CNN knows? The permutation test

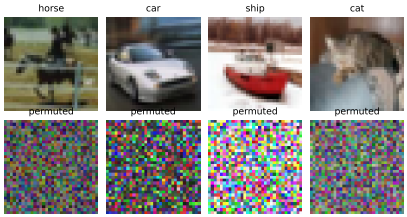


`shuffle_pixels=True`: one fixed random permutation of the 3072 pixel values, applied to every image, training and validation.

Four runs, the same recipe: the CNN and the MLP (`channels=[]`), each with and without the permutation.

- **The MLP**: the permutation relabels the input coordinates; $\mathbf{W}^{(1)}$ absorbs it. Same training problem. If it is hurt, there is a bug.
- **The CNN**: its prior (locality, equivariance) is now *false* about the data; the architecture is unchanged.
- CNN-intact minus CNN-permuted: what the prior is worth. CNN-permuted minus MLP: what is left of it (some true neighbours still land together; pooling still aggregates).

E16: what is it that the CNN knows? The permutation test



`shuffle_pixels=True`: one fixed random permutation of the 3072 pixel values, applied to every image, training and validation.

Four runs, the same recipe: the CNN and the MLP (`channels=[]`), each with and without the permutation.

- **The MLP**: the permutation relabels the input coordinates; $\mathbf{W}^{(1)}$ absorbs it. Same training problem. If it is hurt, there is a bug.
- **The CNN**: its prior (locality, equivariance) is now *false* about the data; the architecture is unchanged.
- CNN-intact minus CNN-permuted: what the prior is worth. CNN-permuted minus MLP: what is left of it (some true neighbours still land together; pooling still aggregates).
- **Predict all four numbers before you run. Write them down.**

E17: width, depth, and where the cost goes

The lab's configurations, before any training ($k = 3$, one conv per block unless stated):

`channels`	params	mult-adds/image
MLP $[512, 256]$	1,707k	1.7M
$[8, 16, 32]$	6.5k	0.8M
$[16, 32, 64]$	24k	2.8M
$[32, 64, 128]$	95k	10M
$[64, 128, 256]$	374k	40M
$[64, 128, 256] \times 2$ convs	1,150k	$\approx$ 130M

Per conv layer: $k^2 C_{\text{in}} C_{\text{out}}$ parameters, $k^2 C_{\text{in}} C_{\text{out}} H_{\text{out}} W_{\text{out}}$ multiply-adds. Doubling every width: $4\times$ both.

E17: width, depth, and where the cost goes

The lab's configurations, before any training ($k = 3$, one conv per block unless stated):

`channels`	params	mult-adds/image
MLP $[512, 256]$	1,707k	1.7M
$[8, 16, 32]$	6.5k	0.8M
$[16, 32, 64]$	24k	2.8M
$[32, 64, 128]$	95k	10M
$[64, 128, 256]$	374k	40M
$[64, 128, 256] \times 2$ convs	1,150k	$\approx$ 130M

Per conv layer: $k^2 C_{\text{in}} C_{\text{out}}$ parameters, $k^2 C_{\text{in}} C_{\text{out}} H_{\text{out}} W_{\text{out}}$ multiply-adds. Doubling every width: $4\times$ both.

- **Width** (channels): more filters per layer, more features per position. Parameters and compute both $\propto C^2$.

E17: width, depth, and where the cost goes

The lab's configurations, before any training ($k = 3$, one conv per block unless stated):

`channels`	params	mult-adds/image
MLP $[512, 256]$	1,707k	1.7M
$[8, 16, 32]$	6.5k	0.8M
$[16, 32, 64]$	24k	2.8M
$[32, 64, 128]$	95k	10M
$[64, 128, 256]$	374k	40M
$[64, 128, 256] \times 2$ convs	1,150k	$\approx$ 130M

Per conv layer: $k^2 C_{\text{in}} C_{\text{out}}$ parameters, $k^2 C_{\text{in}} C_{\text{out}} H_{\text{out}} W_{\text{out}}$ multiply-adds. Doubling every width: $4\times$ both.

- **Width** (channels): more filters per layer, more features per position. Parameters and compute both $\propto C^2$.
- **Depth** (two convs per block): a second nonlinearity at the same resolution, a larger receptive field (5 instead of 3 pixels before the pool), and the parameters that come with it.

E17: width, depth, and where the cost goes

The lab's configurations, before any training ($k = 3$, one conv per block unless stated):

`channels`	params	mult-adds/image
MLP $[512, 256]$	1,707k	1.7M
$[8, 16, 32]$	6.5k	0.8M
$[16, 32, 64]$	24k	2.8M
$[32, 64, 128]$	95k	10M
$[64, 128, 256]$	374k	40M
$[64, 128, 256] \times 2$ convs	1,150k	$\approx$ 130M

Per conv layer: $k^2 C_{\text{in}} C_{\text{out}}$ parameters, $k^2 C_{\text{in}} C_{\text{out}} H_{\text{out}} W_{\text{out}}$ multiply-adds. Doubling every width: $4\times$ both.

- **Width** (channels): more filters per layer, more features per position. Parameters and compute both $\propto C^2$.
- **Depth** (two convs per block): a second nonlinearity at the same resolution, a larger receptive field (5 instead of 3 pixels before the pool), and the parameters that come with it.
- The MLP: most parameters, least compute. The CNN re-uses each parameter at every position: that is where the 1.6M first-layer weights went.

E17: width, depth, and where the cost goes

The lab's configurations, before any training ($k = 3$, one conv per block unless stated):

`channels`	params	mult-adds/image
MLP $[512, 256]$	1,707k	1.7M
$[8, 16, 32]$	6.5k	0.8M
$[16, 32, 64]$	24k	2.8M
$[32, 64, 128]$	95k	10M
$[64, 128, 256]$	374k	40M
$[64, 128, 256] \times 2$ convs	1,150k	$\approx$ 130M

Per conv layer: $k^2 C_{\text{in}} C_{\text{out}}$ parameters, $k^2 C_{\text{in}} C_{\text{out}} H_{\text{out}} W_{\text{out}}$ multiply-adds. Doubling every width: $4\times$ both.

- **Width** (channels): more filters per layer, more features per position. Parameters and compute both $\propto C^2$.
- **Depth** (two convs per block): a second nonlinearity at the same resolution, a larger receptive field (5 instead of 3 pixels before the pool), and the parameters that come with it.
- The MLP: most parameters, least compute. The CNN re-uses each parameter at every position: that is where the 1.6M first-layer weights went.
- **The runs answer**: how many parameters does a CNN need to beat the MLP? Does the gain per parameter keep rising? What does the clock say as the channels double?

E18, and the lab: E16–E18 with cnn_lab.py

```
from cnn_lab import train # same interface, new
    ansatz
train({"epochs": 10}) # E16a: the CNN
train({"epochs": 10, "channels": []}) # E16b: the
    MLP
train({"epochs": 10, "shuffle_pixels": True}) #
    E16c: permuted
train({"epochs": 10, "channels": [],
       "shuffle_pixels": True}) # E16d
train({"epochs": 10, "channels": [64, 128, 256]})
    # E17
train({"epochs": 30, "channels": [...],
       "test": True}) # E18, once
```

Defaults: last week's recipe (AdamW 10^{-3}, decay 0.05, cosine, augmentation), `device="auto"`. ≈ 6 s/epoch on an Apple GPU, 20–60 s on a CPU: **on a CPU run 5 epochs**. Sheet: `lab_cnn.pdf`.

E18, your best CNN, 30 epochs: the protocol, once more.

Predict first, every run. E16 first; E18 if time is short; E17 at home. E19, E20 on Thursday; E21 (EuroSAT) is the homework.

E18, and the lab: E16–E18 with cnn_lab.py

```
from cnn_lab import train # same interface, new
    ansatz
train({"epochs": 10}) # E16a: the CNN
train({"epochs": 10, "channels": []}) # E16b: the
    MLP
train({"epochs": 10, "shuffle_pixels": True}) #
    E16c: permuted
train({"epochs": 10, "channels": [],
       "shuffle_pixels": True}) # E16d
train({"epochs": 10, "channels": [64, 128, 256]})
    # E17
train({"epochs": 30, "channels": [...],
       "test": True}) # E18, once
```

Defaults: last week's recipe (AdamW 10^{-3}, decay 0.05, cosine, augmentation), `device="auto"`. ≈ 6 s/epoch on an Apple GPU, 20–60 s on a CPU: **on a CPU run 5 epochs**. Sheet: `lab_cnn.pdf`.

E18, your best CNN, 30 epochs: the protocol, once more.

- choose the architecture by validation (E17), then *one* run with `test=True`
- with cosine + augmentation, does the validation loss still turn up, and does early stopping still matter?
- training accuracy near 100% with a small gap is a different regime from E4; name it

Predict first, every run. E16 first; E18 if time is short; E17 at home. E19, E20 on Thursday; E21 (EuroSAT) is the homework.

Thursday: transfer, and deeper

E19 (Thursday): take things away

The default is last week's recipe inside the conv net: BatchNorm in every block, augmentation, AdamW with cosine. Remove one at a time.

No BatchNorm

No augmentation

SGD + momentum instead of AdamW

E19 (Thursday): take things away

The default is last week's recipe inside the conv net: BatchNorm in every block, augmentation, AdamW with cosine. Remove one at a time.

No BatchNorm

- last week's depth frame: without normalization the signal through three conv layers is whatever the init gives; the first epochs tell
- watch the first epochs, and whether it recovers

No augmentation

SGD + momentum instead of AdamW

E19 (Thursday): take things away

The default is last week's recipe inside the conv net: BatchNorm in every block, augmentation, AdamW with cosine. Remove one at a time.

No BatchNorm

- last week's depth frame: without normalization the signal through three conv layers is whatever the init gives; the first epochs tell
- watch the first epochs, and whether it recovers

No augmentation

- augmentation is a prior against memorizing. Memorizing needs capacity: a 95k net at 10 epochs may not have enough to memorize $45{,}000$ images yet
- the signal is the gap: training accuracy, and whether the validation loss turns up (E4). Expect the answer to depend on the net's size and on the number of epochs

SGD + momentum instead of AdamW

E19 (Thursday): take things away

The default is last week's recipe inside the conv net: BatchNorm in every block, augmentation, AdamW with cosine. Remove one at a time.

No BatchNorm

- last week's depth frame: without normalization the signal through three conv layers is whatever the init gives; the first epochs tell
- watch the first epochs, and whether it recovers

No augmentation

- augmentation is a prior against memorizing. Memorizing needs capacity: a 95k net at 10 epochs may not have enough to memorize $45{,}000$ images yet
- the signal is the gap: training accuracy, and whether the validation loss turns up (E4). Expect the answer to depend on the net's size and on the number of epochs

SGD + momentum instead of AdamW

- the classic CNN recipe: SGD, momentum 0.9, $\eta = 0.05$, decay 5×10^{-4}. Most published CIFAR numbers use it
- two legitimate recipes; neither is right in general. Learn to read which one is winning from the curves, not the folklore

E19 (Thursday): take things away

The default is last week's recipe inside the conv net: BatchNorm in every block, augmentation, AdamW with cosine. Remove one at a time.

No BatchNorm

- last week's depth frame: without normalization the signal through three conv layers is whatever the init gives; the first epochs tell
- watch the first epochs, and whether it recovers

No augmentation

- augmentation is a prior against memorizing. Memorizing needs capacity: a 95k net at 10 epochs may not have enough to memorize $45{,}000$ images yet
- the signal is the gap: training accuracy, and whether the validation loss turns up (E4). Expect the answer to depend on the net's size and on the number of epochs

SGD + momentum instead of AdamW

- the classic CNN recipe: SGD, momentum 0.9, $\eta = 0.05$, decay 5×10^{-4}. Most published CIFAR numbers use it
- two legitimate recipes; neither is right in general. Learn to read which one is winning from the curves, not the folklore

Before each run: which removal costs the most, and in which epoch does it show?

E20 (Thursday, warm-up): transfer within CIFAR-10

```
train({"classes": [0,1,2,3,4], "epochs": 10,
      "save_weights": "pre.pt"}) # (a) pretrain
for n in (10, 100, 1000): # per class
  train({"classes": [5,6,7,8,9],
        "n_per_class": n}) # (b) scratch
  train({..., "init_from": "pre.pt", "freeze":
       True,
         "lr": 1e-2, "batch_size": 32, "epochs":
              100,
         "augment": False}) # (c) frozen + head
  train({..., "init_from": "pre.pt"}) # (d) fine-
       tune
```

The EuroSAT experiment in miniature, under control: the same image statistics, disjoint classes, no ImageNet. One minute on a GPU, five on a CPU.

E20 (Thursday, warm-up): transfer within CIFAR-10

```
train({"classes": [0,1,2,3,4], "epochs": 10,
      "save_weights": "pre.pt"}) # (a) pretrain
for n in (10, 100, 1000): # per class
  train({"classes": [5,6,7,8,9],
        "n_per_class": n}) # (b) scratch
  train({..., "init_from": "pre.pt", "freeze":
       True,
         "lr": 1e-2, "batch_size": 32, "epochs":
              100,
         "augment": False}) # (c) frozen + head
  train({..., "init_from": "pre.pt"}) # (d) fine-
       tune
```

The EuroSAT experiment in miniature, under control: the same image statistics, disjoint classes, no ImageNet. One minute on a GPU, five on a CPU.

- **Scratch**: 95k parameters from 50 images. The baseline that must fail first.

E20 (Thursday, warm-up): transfer within CIFAR-10

```
train({"classes": [0,1,2,3,4], "epochs": 10,
      "save_weights": "pre.pt"}) # (a) pretrain
for n in (10, 100, 1000): # per class
  train({"classes": [5,6,7,8,9],
        "n_per_class": n}) # (b) scratch
  train({..., "init_from": "pre.pt", "freeze":
       True,
         "lr": 1e-2, "batch_size": 32, "epochs":
              100,
         "augment": False}) # (c) frozen + head
  train({..., "init_from": "pre.pt"}) # (d) fine-
       tune
```

The EuroSAT experiment in miniature, under control: the same image statistics, disjoint classes, no ImageNet. One minute on a GPU, five on a CPU.

- **Scratch**: 95k parameters from 50 images. The baseline that must fail first.
- **Frozen + head**: the conv blocks never saw a dog or a ship; 645 numbers train. A linear head on 128 features needs its own recipe: a larger η, many epochs (the default cosine over 10 epochs gives it ten steps).

E20 (Thursday, warm-up): transfer within CIFAR-10

```
train({"classes": [0,1,2,3,4], "epochs": 10,
       "save_weights": "pre.pt"}) # (a) pretrain
for n in (10, 100, 1000): # per class
  train({"classes": [5,6,7,8,9],
         "n_per_class": n}) # (b) scratch
  train({..., "init_from": "pre.pt", "freeze":
       True,
         "lr": 1e-2, "batch_size": 32, "epochs":
              100,
         "augment": False}) # (c) frozen + head
  train({..., "init_from": "pre.pt"}) # (d) fine-
       tune
```

The EuroSAT experiment in miniature, under control: the same image statistics, disjoint classes, no ImageNet. One minute on a GPU, five on a CPU.

- **Scratch**: 95k parameters from 50 images. The baseline that must fail first.
- **Frozen + head**: the conv blocks never saw a dog or a ship; 645 numbers train. A linear head on 128 features needs its own recipe: a larger η, many epochs (the default cosine over 10 epochs gives it ten steps).
- **Fine-tune**: everything, from the pretrained start. With 50 images and the default η it can undo what it inherited.

E20 (Thursday, warm-up): transfer within CIFAR-10

```
train({"classes": [0,1,2,3,4], "epochs": 10,
       "save_weights": "pre.pt"}) # (a) pretrain
for n in (10, 100, 1000): # per class
  train({"classes": [5,6,7,8,9],
         "n_per_class": n}) # (b) scratch
  train({..., "init_from": "pre.pt", "freeze":
       True,
         "lr": 1e-2, "batch_size": 32, "epochs":
              100,
         "augment": False}) # (c) frozen + head
  train({..., "init_from": "pre.pt"}) # (d) fine-
       tune
```

The EuroSAT experiment in miniature, under control: the same image statistics, disjoint classes, no ImageNet. One minute on a GPU, five on a CPU.

- **Scratch**: 95k parameters from 50 images. The baseline that must fail first.
- **Frozen + head**: the conv blocks never saw a dog or a ship; 645 numbers train. A linear head on 128 features needs its own recipe: a larger η, many epochs (the default cosine over 10 epochs gives it ten steps).
- **Fine-tune**: everything, from the pretrained start. With 50 images and the default η it can undo what it inherited.
- Predict the three curves vs. n before running; then the same on EuroSAT, with a real backbone.

E21 (homework): transfer learning on EuroSAT

```
from eurosat_lab import train # same interface
for n in (100, 300, 1000): # labelled images
  train({"method": "mlp", "n": n})
  train({"method": "scratch", "n": n})
  train({"method": "frozen", "n": n})
  train({"method": "finetune", "n": n})
```

EuroSAT: $27{,}000$ Sentinel-2 patches, 64×64, 10 land-use classes; $2{,}000$ validation, $5{,}000$ test, n labelled for training. The ImageNet ResNet-18 comes with `torchvision`.

E21 (homework): transfer learning on EuroSAT

```
from eurosat_lab import train # same interface
for n in (100, 300, 1000): # labelled images
  train({"method": "mlp", "n": n})
  train({"method": "scratch", "n": n})
  train({"method": "frozen", "n": n})
  train({"method": "finetune", "n": n})
```

EuroSAT: $27{,}000$ Sentinel-2 patches, 64×64, 10 land-use classes; $2{,}000$ validation, $5{,}000$ test, n labelled for training. The ImageNet ResNet-18 comes with `torchvision`.

- **mlp**, **scratch**: the two ways we know. The baseline that must fail first at $n = 100$; and does a ResNet from scratch do better than an MLP with 10 images per class?

E21 (homework): transfer learning on EuroSAT

```
from eurosat_lab import train # same interface
for n in (100, 300, 1000): # labelled images
  train({"method": "mlp", "n": n})
  train({"method": "scratch", "n": n})
  train({"method": "frozen", "n": n})
  train({"method": "finetune", "n": n})
```

EuroSAT: $27{,}000$ Sentinel-2 patches, 64×64, 10 land-use classes; $2{,}000$ validation, $5{,}000$ test, n labelled for training. The ImageNet ResNet-18 comes with `torchvision`.

- **mlp**, **scratch**: the two ways we know. The baseline that must fail first at $n = 100$; and does a ResNet from scratch do better than an MLP with 10 images per class?
- **frozen**: the backbone never saw a satellite image; $5{,}130$ numbers train. If this works, what ImageNet taught it was not about ImageNet.

E21 (homework): transfer learning on EuroSAT

```
from eurosat_lab import train # same interface
for n in (100, 300, 1000): # labelled images
  train({"method": "mlp", "n": n})
  train({"method": "scratch", "n": n})
  train({"method": "frozen", "n": n})
  train({"method": "finetune", "n": n})
```

EuroSAT: $27{,}000$ Sentinel-2 patches, 64×64, 10 land-use classes; $2{,}000$ validation, $5{,}000$ test, n labelled for training. The ImageNet ResNet-18 comes with `torchvision`.

- **mlp**, **scratch**: the two ways we know. The baseline that must fail first at $n = 100$; and does a ResNet from scratch do better than an MLP with 10 images per class?
- **frozen**: the backbone never saw a satellite image; $5{,}130$ numbers train. If this works, what ImageNet taught it was not about ImageNet.
- **finetune**: everything, from the pretrained start, small learning rate. Expect it to overtake the frozen features as n grows.

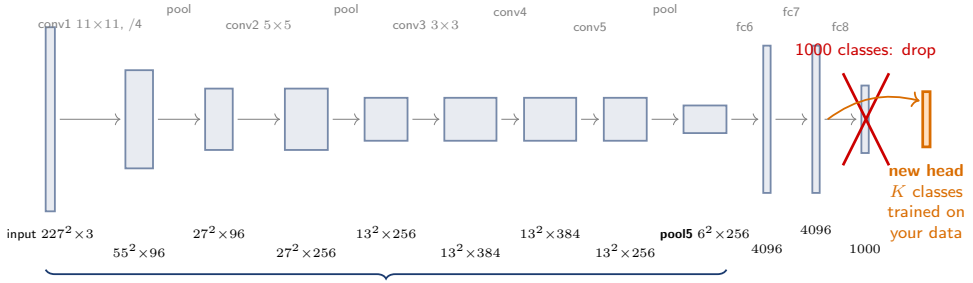
E21 (homework): transfer learning on EuroSAT

```
from eurosat_lab import train # same interface
for n in (100, 300, 1000): # labelled images
  train({"method": "mlp", "n": n})
  train({"method": "scratch", "n": n})
  train({"method": "frozen", "n": n})
  train({"method": "finetune", "n": n})
```

EuroSAT: $27{,}000$ Sentinel-2 patches, 64×64, 10 land-use classes; $2{,}000$ validation, $5{,}000$ test, n labelled for training. The ImageNet ResNet-18 comes with `torchvision`.

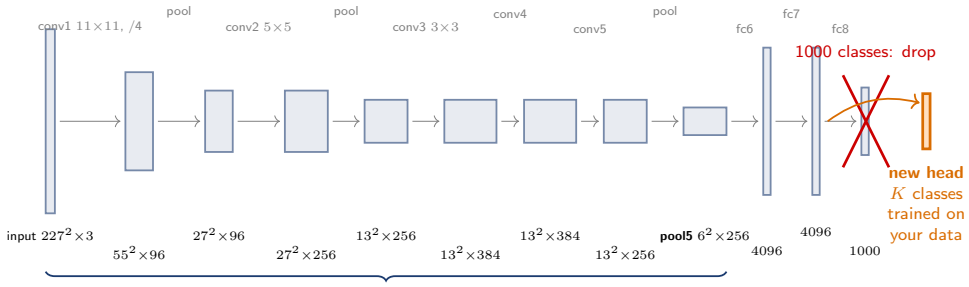
- **mlp**, **scratch**: the two ways we know. The baseline that must fail first at $n = 100$; and does a ResNet from scratch do better than an MLP with 10 images per class?
- **frozen**: the backbone never saw a satellite image; $5{,}130$ numbers train. If this works, what ImageNet taught it was not about ImageNet.
- **finetune**: everything, from the pretrained start, small learning rate. Expect it to overtake the frozen features as n grows.
- Predict the four curves vs. n before running. On a CPU: **frozen** and **mlp** at every n; **scratch** and **finetune** at $n \leq 300$, or with `size=64`.

Transfer learning: keep the backbone, replace the head



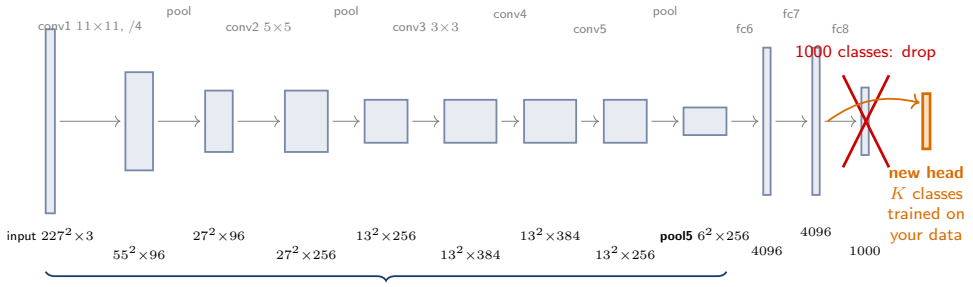
backbone: trained on ImageNet, 1.3M photos. Keep it.
frozen (features only) or fine-tuned (small η)

Transfer learning: keep the backbone, replace the head



- The stencils and the deep features are about images in general (the two AlexNet figures); only the last layer is about the 1000 ImageNet classes. Cut it off, add a linear layer with K outputs, train on your n images: the head only when n is small, everything with a small learning rate when n is larger.

Transfer learning: keep the backbone, replace the head



- The stencils and the deep features are about images in general (the two AlexNet figures); only the last layer is about the 1000 ImageNet classes. Cut it off, add a linear layer with K outputs, train on your n images: the head only when n is small, everything with a small learning rate when n is larger.
- The same move everywhere: a language model pretrained on the web, then a head or a light fine-tune for your task; speech, protein and molecule models alike. *Pretrain on what is plentiful, adapt on what is scarce.*

Borrow a trained network: transfer learning

A ResNet-18 trained on ImageNet (1.3M photos, 1000 classes) has learned edges, textures, parts. Keep the **backbone**, replace the **head**:

```
from torchvision.models import resnet18
m = resnet18(weights="IMAGENET1K_V1")
m.requires_grad_(False) # frozen: features
    only
m.fc = nn.Linear(512, K) # a new head for K
    classes
# fine-tuning: no freeze; AdamW lr 1e-4, few
    epochs
```

Borrow a trained network: transfer learning

A ResNet-18 trained on ImageNet (1.3M photos, 1000 classes) has learned edges, textures, parts. Keep the **backbone**, replace the **head**:

```
from torchvision.models import resnet18
m = resnet18(weights="IMAGENET1K_V1")
m.requires_grad_(False) # frozen: features
    only
m.fc = nn.Linear(512, K) # a new head for K
    classes
# fine-tuning: no freeze; AdamW lr 1e-4, few
    epochs
```

- **Frozen features**: the backbone once per image, a linear classifier on the 512 numbers. Seconds; 10 images per class suffice.

Borrow a trained network: transfer learning

A ResNet-18 trained on ImageNet (1.3M photos, 1000 classes) has learned edges, textures, parts. Keep the **backbone**, replace the **head**:

```
from torchvision.models import resnet18
m = resnet18(weights="IMAGENET1K_V1")
m.requires_grad_(False) # frozen: features
    only
m.fc = nn.Linear(512, K) # a new head for K
    classes
# fine-tuning: no freeze; AdamW lr 1e-4, few
    epochs
```

- **Frozen features**: the backbone once per image, a linear classifier on the 512 numbers. Seconds; 10 images per class suffice.
- **Fine-tuning**: train everything, small η, augmentation. Better with a few hundred images or more.

Borrow a trained network: transfer learning

A ResNet-18 trained on ImageNet (1.3M photos, 1000 classes) has learned edges, textures, parts. Keep the **backbone**, replace the **head**:

```
from torchvision.models import resnet18
m = resnet18(weights="IMAGENET1K_V1")
m.requires_grad_(False) # frozen: features
    only
m.fc = nn.Linear(512, K) # a new head for K
    classes
# fine-tuning: no freeze; AdamW lr 1e-4, few
    epochs
```

- **Frozen features**: the backbone once per image, a linear classifier on the 512 numbers. Seconds; 10 images per class suffice.
- **Fine-tuning**: train everything, small η, augmentation. Better with a few hundred images or more.

EuroSAT: Sentinel-2 satellite patches, 64×64, 10 land-use classes; n labelled images, test on 5,000:

n	100	300	1,000	3,000
MLP, pixels	21	34	46	50
ResNet-18, scratch	48	64	74	85
frozen + head	78	85	90	92
fine-tuned	80	91	95	96

Borrow a trained network: transfer learning

A ResNet-18 trained on ImageNet (1.3M photos, 1000 classes) has learned edges, textures, parts. Keep the **backbone**, replace the **head**:

```
from torchvision.models import resnet18
m = resnet18(weights="IMAGENET1K_V1")
m.requires_grad_(False) # frozen: features
    only
m.fc = nn.Linear(512, K) # a new head for K
    classes
# fine-tuning: no freeze; AdamW lr 1e-4, few
    epochs
```

- **Frozen features**: the backbone once per image, a linear classifier on the 512 numbers. Seconds; 10 images per class suffice.
- **Fine-tuning**: train everything, small η, augmentation. Better with a few hundred images or more.

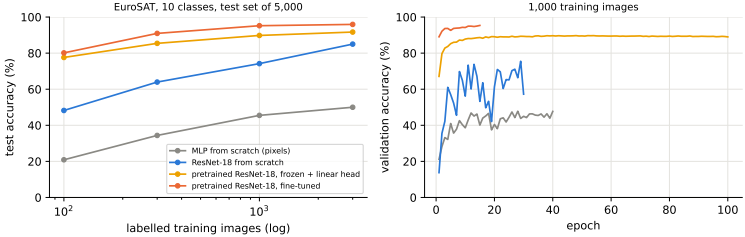
EuroSAT: Sentinel-2 satellite patches, 64×64, 10 land-use classes; n labelled images, test on 5,000:

n	100	300	1,000	3,000
MLP, pixels	21	34	46	50
ResNet-18, scratch	48	64	74	85
frozen + head	78	85	90	92
fine-tuned	80	91	95	96

Key point

With 10 images per class the borrowed features give 78%. For a small scientific image set, try this before training anything.

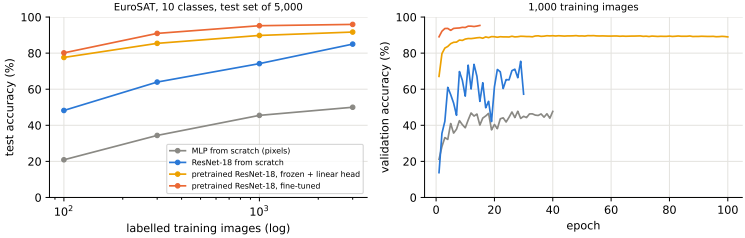
EuroSAT, the curves



- **EuroSAT** (Helber et al. 2019): $27{,}000$ Sentinel-2 satellite patches, 64×64 RGB, 10 land-use classes (crop, forest, highway, industrial, residential, river, sea, ...). Our demo: n labelled images to train, $5{,}000$ to test; four classifiers: the session-2 MLP on the pixels, ResNet-18 from scratch, ImageNet ResNet-18 frozen + a linear head, the same fine-tuned.

`demos/transfer_eurosat.py`, ≈ 10 min on a laptop GPU; the workshop starts from it.

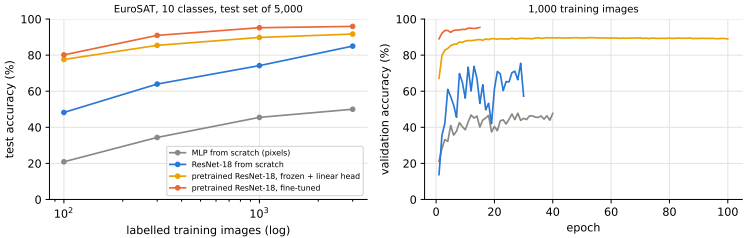
EuroSAT, the curves



- **EuroSAT** (Helber et al. 2019): $27{,}000$ Sentinel-2 satellite patches, 64×64 RGB, 10 land-use classes (crop, forest, highway, industrial, residential, river, sea, ...). Our demo: n labelled images to train, $5{,}000$ to test; four classifiers: the session-2 MLP on the pixels, ResNet-18 from scratch, ImageNet ResNet-18 frozen + a linear head, the same fine-tuned.
- Satellite patches are not ImageNet photos; the features transfer anyway (and to microscopy, histology, galaxies, spectrograms drawn as images).

`demos/transfer_eurosat.py`, ≈ 10 min on a laptop GPU; the workshop starts from it.

EuroSAT, the curves



- **EuroSAT** (Helber et al. 2019): $27{,}000$ Sentinel-2 satellite patches, 64×64 RGB, 10 land-use classes (crop, forest, highway, industrial, residential, river, sea, ...). Our demo: n labelled images to train, $5{,}000$ to test; four classifiers: the session-2 MLP on the pixels, ResNet-18 from scratch, ImageNet ResNet-18 frozen + a linear head, the same fine-tuned.
- Satellite patches are not ImageNet photos; the features transfer anyway (and to microscopy, histology, galaxies, spectrograms drawn as images).
- Not to data that are not images: spectra, time series, graphs. There, pretrain on your own unlabelled data: the workshop's experiment.

`demos/transfer_eurosat.py`, ≈ 10 min on a laptop GPU; the workshop starts from it.

Don’t be a hero

In practice: use whatever works best on ImageNet. If you’re feeling a bit of a fatigue in thinking about the architectural decisions, you’ll be pleased to know that in 90% or more of applications you should not have to worry about these. I like to summarize this point as “*don’t be a hero*”: Instead of rolling your own architecture for a problem, you should look at whatever architecture currently works best on ImageNet, download a pretrained model and finetune it on your data. You should rarely ever have to train a ConvNet from scratch or design one from scratch. I also made this point at the Deep Learning school.

Andrej Karpathy, CS231n course notes (Stanford), “Convolutional Networks”.

Don't be a hero

In practice: use whatever works best on ImageNet. If you're feeling a bit of a fatigue in thinking about the architectural decisions, you'll be pleased to know that in 90% or more of applications you should not have to worry about these. I like to summarize this point as "*don't be a hero*": Instead of rolling your own architecture for a problem, you should look at whatever architecture currently works best on ImageNet, download a pretrained model and finetune it on your data. You should rarely ever have to train a ConvNet from scratch or design one from scratch. I also made this point at the Deep Learning school.

Andrej Karpathy, CS231n course notes (Stanford), "Convolutional Networks".

- For images, the backbone is a solved problem: take the current best on ImageNet, pretrained, and fine-tune. Your effort goes into the data, the labels, the validation protocol, and the head.

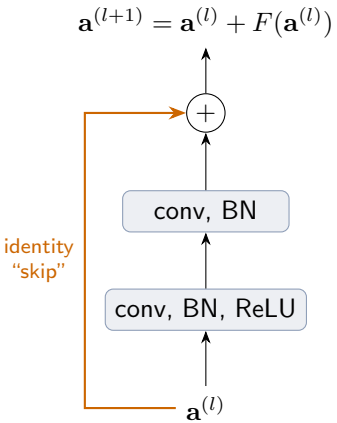
Don't be a hero

In practice: use whatever works best on ImageNet. If you're feeling a bit of a fatigue in thinking about the architectural decisions, you'll be pleased to know that in 90% or more of applications you should not have to worry about these. I like to summarize this point as "*don't be a hero*": Instead of rolling your own architecture for a problem, you should look at whatever architecture currently works best on ImageNet, download a pretrained model and finetune it on your data. You should rarely ever have to train a ConvNet from scratch or design one from scratch. I also made this point at the Deep Learning school.

Andrej Karpathy, CS231n course notes (Stanford), "Convolutional Networks".

- For images, the backbone is a solved problem: take the current best on ImageNet, pretrained, and fine-tune. Your effort goes into the data, the labels, the validation protocol, and the head.
- The exception is data that are not photographs: then the workshop's question, what to pretrain on, is yours to answer.

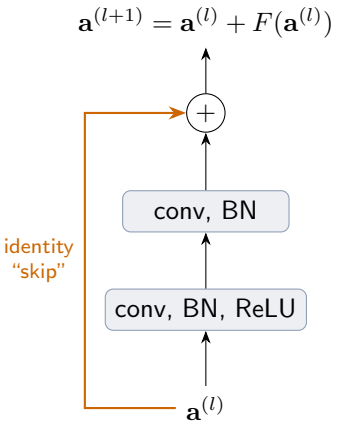
Going deep: skip connections



He, Zhang, Ren, Sun 2016 (ResNet).

- Last week: through depth, signal and gradient are a *product* of one factor per layer. Add the identity around every block and the product becomes a *sum*: the gradient reaches layer 1 through the skips, whatever the blocks do.

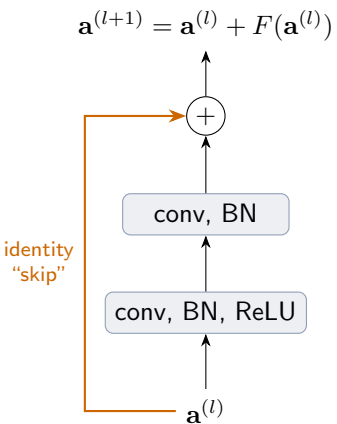
Going deep: skip connections



He, Zhang, Ren, Sun 2016 (ResNet).

- Last week: through depth, signal and gradient are a *product* of one factor per layer. Add the identity around every block and the product becomes a *sum*: the gradient reaches layer 1 through the skips, whatever the blocks do.
- The block learns a *residual* F; $F = 0$ is easy, so deeper is never worse in practice. 18, 50, 152 layers.

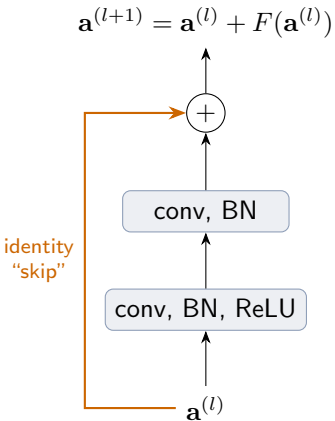
Going deep: skip connections



He, Zhang, Ren, Sun 2016 (ResNet).

- Last week: through depth, signal and gradient are a *product* of one factor per layer. Add the identity around every block and the product becomes a *sum*: the gradient reaches layer 1 through the skips, whatever the blocks do.
- The block learns a *residual* F; $F = 0$ is easy, so deeper is never worse in practice. 18, 50, 152 layers.
- **ResNet-18**: $[64, 128, 256, 512]$ channels, two blocks each, 11M parameters; 94–95% on CIFAR-10 from scratch, 70% top-1 on ImageNet. The standard backbone.

Going deep: skip connections



He, Zhang, Ren, Sun 2016 (ResNet).

- Last week: through depth, signal and gradient are a *product* of one factor per layer. Add the identity around every block and the product becomes a *sum*: the gradient reaches layer 1 through the skips, whatever the blocks do.
- The block learns a *residual* F; $F = 0$ is easy, so deeper is never worse in practice. 18, 50, 152 layers.
- **ResNet-18**: $[64, 128, 256, 512]$ channels, two blocks each, 11M parameters; 94–95% on CIFAR-10 from scratch, 70% top-1 on ImageNet. The standard backbone.
- The same trick everywhere: transformers are residual stacks (week 6). Inside: BatchNorm, global average pooling, last week's recipe.

Part I, the map

failure	what we measured	what fixed it
optimization	$\ln 10$ forever; crawl, oscillate, diverge; the floor	normalization, momentum/Adam, schedules, init, BatchNorm (week 2)
generalization	training 100%, validation 55	priors: augmentation, decay, early stopping, dropout (week 2)
approximation	$\approx 60\%$ for the MLP, whatever we did	the ansatz: convolution, pooling, skips; a borrowed backbone (today)

Part I, the map

failure	what we measured	what fixed it
optimization	$\ln 10$ forever; crawl, oscillate, diverge; the floor	normalization, momentum/Adam, schedules, init, BatchNorm (week 2)
generalization	training 100%, validation 55	priors: augmentation, decay, early stopping, dropout (week 2)
approximation	$\approx 60\%$ for the MLP, whatever we did	the ansatz: convolution, pooling, skips; a borrowed backbone (today)

- **Thursday 29 Oct**: going deep (skip connections), and borrowing a trained network (transfer learning); in class E19 and E20. **Homework**: E21, transfer learning on EuroSAT, a satellite image set, with `eurosat_lab.py`: run `python eurosat_lab.py --download` at home before Thursday (140 MB).

Part I, the map

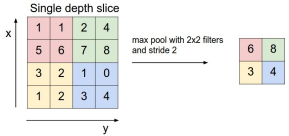
failure	what we measured	what fixed it
optimization	$\ln 10$ forever; crawl, oscillate, diverge; the floor	normalization, momentum/Adam, schedules, init, BatchNorm (week 2)
generalization	training 100%, validation 55	priors: augmentation, decay, early stopping, dropout (week 2)
approximation	$\approx 60\%$ for the MLP, whatever we did	the ansatz: convolution, pooling, skips; a borrowed backbone (today)

- **Thursday 29 Oct**: going deep (skip connections), and borrowing a trained network (transfer learning); in class E19 and E20. **Homework**: E21, transfer learning on EuroSAT, a satellite image set, with `eurosat_lab.py`: run `python eurosat_lab.py --download` at home before Thursday (140 MB).
- Then: autoencoders (Mon 2 Nov), VAEs and anomaly detection (Thu 5 Nov).

Backup

Convolution arithmetic

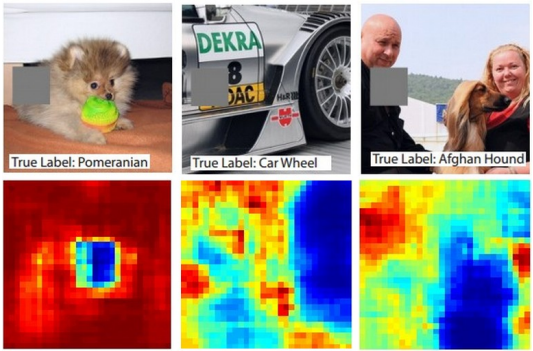
- Output size of a conv with kernel k, padding p, stride s on an input of size H: $\lfloor (H + 2p - k)/s \rfloor + 1$. "Same" padding $p = \lfloor k/2 \rfloor$ with $s = 1$ keeps H (the 1-D figure: $7 \to 5$ with $p = 0$, $\to 3$ with $s = 2$).
- Parameters per conv layer: $k^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}$. Multiply-adds: $k^2 C_{\text{in}} C_{\text{out}}\, H_{\text{out}} W_{\text{out}}$. The lab's default: $896 + 18{,}496 + 73{,}856$ conv weights; the $32 \to 64$ layer on 16×16 costs 4.7M multiply-adds per image.
- Receptive field after L layers of 3×3 convs with stride 1: $2L + 1$ pixels; each 2×2 pool doubles the step of everything after it. The lab's network: $3 \to 7 \to 15$ pixels, then global pooling sees the whole 32×32.
- Transposed (upsampling), dilated, depthwise-separable, 1×1: variations on the operator. 1-D for signals and spectra; 3-D for volumes; graph convolutions (the same weights at every node, over the neighbours).



Pooling acts on every depth slice independently.

Figure: CS231n course notes, Stanford (A. Karpathy et al.).

What the network looks at

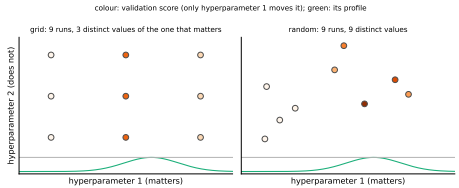


Zeiler & Fergus 2014, Fig. 7; via CS231n.

- Slide a grey square over the image and record the probability of the true class at each position (bottom row, blue = low).
- The dog's face, the wheel, the people's faces: the evidence is local, and the network found it without boxes or labels for parts.
- The simplest diagnostic of what a trained classifier uses; for scientific images, the first sanity check against shortcuts (a scale bar, a date stamp, a detector's edge).

The protocol and the search (for the workshop)

- **The unit is the seed spread.** E1: 1.2 points in `val_acc_at_best`, 3.2 in the last epoch. $+1$ needs three seeds of both configs; $+5$ needs one.
- **Random beats grid.** k hyperparameters, one matters: a grid of m^k runs tries m values of it, random tries m^k. Log-uniform for η, λ, widths.
- **Order**: η first (3 per decade); then λ and dropout; then width, depth; batch size by memory; the rest at defaults. Short runs first, the top few to full length.
- **Leaks**: normalization statistics from the training set only; split *by group* when samples are correlated (patient, star, sample, run, day); time series by time; the test set once. Small data: k-fold cross-validation.



Bergstra & Bengio 2012, Fig. 1.

Transfer learning: when it does not work, and details that bite

- Input is not an image, or far from natural-image statistics (raw interferograms, 1-D spectra): frozen ImageNet features are no better than random features; fine-tuning is at best a slightly better initialization. Pretrain on your own unlabelled data instead (self-supervised; the workshop's half-labels experiment is the simplest version).
- Different channels (13 Sentinel bands, grayscale): a new first layer (average the RGB filters for grayscale).
- ImageNet normalization of the input ($\mu = (0.485, 0.456, 0.406)$, $\sigma = (0.229, 0.224, 0.225)$); resize small images up ($64 \to 128$ helped here); BatchNorm in `eval()` for frozen features, `train()` with batch ≥ 16 for fine-tuning; a $10\times$ smaller η for the backbone than for the head.
- Tiny n: k-NN on the frozen features as the baseline before any training.
- Published EuroSAT from scratch with all $21{,}600$ training images: $\approx 98\%$ (Helber et al. 2019); our fine-tuned ResNet-18 with $3{,}000$: 96%.